```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>For You...</title>
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <!-- Ultra-modern font pairing: Outfit (geometric sans) and Playfair Display (elegant serif) -->
    <link
href="https://fonts.googleapis.com/css2?family=Outfit:wght@300;500;800&family=Playfair+Display:
ital,wght@1,500;1,700&display=swap" rel="stylesheet">


    <style>
        :root {
            /* 2026 Color Palette */
            --bg-deep: #050507;
            --bg-gradient: linear-gradient(145deg, #050507 0%, #120b18 50%, #050507 100%);
            --accent-glow: #8a2be2;
            --accent-secondary: #ff2a85;
            --text-main: #f0f0f5;
            --text-muted: #a0a0b5;
            --glass-bg: rgba(255, 255, 255, 0.03);
            --glass-border: rgba(255, 255, 255, 0.08);
            --ease-out-expo: cubic-bezier(0.16, 1, 0.3, 1);
        }

        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }
```

```
body, html {
    width: 100%;
    background-color: var(--bg-deep);
    color: var(--text-main);
    font-family: 'Outfit', sans-serif;
    overflow-x: hidden;
    scroll-behavior: smooth;
}

/* Ambient Noise Texture */
.noise-overlay {
    position: fixed;
    top: 0; left: 0; width: 100vw; height: 100vh;
    pointer-events: none;
    z-index: 999;
    opacity: 0.04;
    background: url('data:image/svg+xml;utf8,%3Csvg viewBox="0 0 200 200"
xmlns="http://www.w3.org/2000/svg"%3E%3Cfilter id="noiseFilter"%3E%3CfeTurbulence
type="fractalNoise" baseFrequency="0.65" numOctaves="3"
stitchTiles="stitch"/%3E%3C/filter%3E%3Crect width="100%25" height="100%25"
filter="url(%23noiseFilter)"/%3E%3C/svg%3E');
}

/* Background Canvas */
#ambient-canvas {
    position: fixed;
    top: 0; left: 0;
    width: 100vw; height: 100vh;
    z-index: 0;
    pointer-events: none;
}
```

```
/* Layout & Sections */
.section {
    position: relative;
    width: 100%;
    min-height: 100vh;
    display: flex;
    align-items: center;
    justify-content: center;
    z-index: 10;
    padding: 2rem;
}

.container {
    max-width: 800px;
    text-align: center;
    display: flex;
    flex-direction: column;
    gap: 2rem;
}

/* Typography */
h1 {
    font-size: clamp(3rem, 8vw, 6rem);
    font-weight: 800;
    line-height: 1.1;
    letter-spacing: -0.03em;
    background: linear-gradient(to right, #fff, #a0a0b5);
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    margin-bottom: 1rem;
```

```
}

.serif-italic {
    font-family: 'Playfair Display', serif;
    font-style: italic;
    font-weight: 500;
    color: var(--accent-secondary);
    font-size: clamp(2rem, 5vw, 4rem);
}

p {
    font-size: clamp(1.2rem, 2.5vw, 1.8rem);
    font-weight: 300;
    color: var(--text-muted);
    line-height: 1.6;
}

/* Glassmorphism Cards */
.glass-card {
    background: var(--glass-bg);
    backdrop-filter: blur(16px);
    -webkit-backdrop-filter: blur(16px);
    border: 1px solid var(--glass-border);
    border-radius: 24px;
    padding: 3rem 2rem;
    box-shadow: 0 30px 60px rgba(0, 0, 0, 0.4);
    transition: transform 0.4s var(--ease-out-expo), box-shadow 0.4s var(--ease-out-expo);
}

.glass-card:hover {
    transform: translateY(-10px);
```

```
    box-shadow: 0 40px 80px rgba(138, 43, 226, 0.15);
    border: 1px solid rgba(255, 255, 255, 0.15);
}

/* Scroll Reveal Animations */
.reveal {
    opacity: 0;
    transform: translateY(60px);
    transition: all 1.2s var(--ease-out-expo);
}

.reveal.active {
    opacity: 1;
    transform: translateY(0);
}

/* Modern Buttons */
.btn {
    appearance: none;
    background: transparent;
    border: 1px solid var(--glass-border);
    color: var(--text-main);
    padding: 1rem 2.5rem;
    font-size: 1.2rem;
    font-family: 'Outfit', sans-serif;
    border-radius: 50px;
    cursor: pointer;
    position: relative;
    overflow: hidden;
    transition: all 0.4s ease;
    margin-top: 2rem;
```

```
}

.btn::before {
    content: '';
    position: absolute;
    top: 0; left: -100%;
    width: 100%; height: 100%;
    background: linear-gradient(90deg, transparent, rgba(255,255,255,0.2), transparent);
    transition: all 0.6s ease;
}

.btn:hover {
    background: rgba(255, 255, 255, 0.05);
    border-color: var(--accent-glow);
    box-shadow: 0 0 20px rgba(138, 43, 226, 0.4);
}

.btn:hover::before {
    left: 100%;
}

/* Final Scene Specifics */
.glow-text {
    text-shadow: 0 0 30px rgba(255, 42, 133, 0.6), 0 0 60px rgba(138, 43, 226, 0.4);
}

/* Scroll indicator */
.scroll-indicator {
    position: absolute;
    bottom: 40px;
    left: 50%;
```

```
    transform: translateX(-50%);
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 10px;
    opacity: 0.5;
    animation: bounce 2s infinite ease-in-out;
}

.scroll-indicator div {
    width: 1px;
    height: 40px;
    background: linear-gradient(to bottom, var(--text-main), transparent);
}

@keyframes bounce {
    0%, 100% { transform: translate(-50%, 0); }
    50% { transform: translate(-50%, 10px); }
}

/* Initial Entry Screen */
#entry-screen {
    position: fixed;
    top: 0; left: 0; width: 100%; height: 100%;
    background: var(--bg-deep);
    z-index: 100;
    display: flex;
    align-items: center;
    justify-content: center;
    transition: opacity 1s var(--ease-out-expo);
}
```

```
        #entry-screen.hidden {
            opacity: 0;
            pointer-events: none;
        }
    </style>
</head>
<body>

    <!-- Ambient Textures and Canvas -->
    <div class="noise-overlay"></div>
    <canvas id="ambient-canvas"></canvas>

    <!-- Entry Screen (Ensures user interaction to start animations/story) -->
    <div id="entry-screen">
        <button class="btn" id="start-btn">I have something for you...</button>
    </div>

    <!-- The Journey -->
    <main id="story-container" style="display: none;">

        <!-- Scene 1 -->
        <section class="section">
            <div class="container reveal">
                <p>Every once in a while, the universe aligns perfectly...</p>
                <div class="scroll-indicator">
                    <span>Scroll gently</span>
                    <div></div>
                </div>
            </div>
        </section>
```

```
<!-- Scene 2 -->
<section class="section">
  <div class="container glass-card reveal">
    <h2 class="serif-italic">Just to create someone extraordinary.</h2>
    <p>Someone whose energy shifts the atmosphere. Whose smile can instantly turn a bad day into a beautiful one.</p>
  </div>
</section>

<!-- Scene 3 -->
<section class="section">
  <div class="container reveal">
    <p>In a world full of billions of people,</p>
    <h1>I'm so incredibly <br><span class="serif-italic">grateful</span></h1>
    <p>that our paths crossed.</p>
  </div>
</section>

<!-- Scene 4 -->
<section class="section">
  <div class="container glass-card reveal">
    <p>You inspire me, you bring light into my life, and you make every single moment better just by being you.</p>
    <p style="margin-top: 1rem; font-size: 1rem;">(And honestly, you look absolutely stunning today.)</p>
  </div>
</section>

<!-- Finale -->
<section class="section">
  <div class="container reveal">
```

```
            <p>So today, we celebrate the masterpiece that is you.</p>
            <h1 class="glow-text" style="font-size: clamp(4rem, 10vw, 8rem); margin-top: 2rem;">Happy<br>Birthday.</h1>
            <button class="btn" id="wish-btn" style="margin-top: 3rem;">Make a Wish ✨</button>
        </div>
    </section>

</main>

<script>
    /* --- Initialization & Entry --- */
    const entryScreen = document.getElementById('entry-screen');
    const startBtn = document.getElementById('start-btn');
    const storyContainer = document.getElementById('story-container');

    startBtn.addEventListener('click', () => {
        entryScreen.classList.add('hidden');
        setTimeout(() => {
            storyContainer.style.display = 'block';
            initScrollReveal();
        }, 500);
    });

    /* --- Intersection Observer for Scroll Storytelling --- */
    function initScrollReveal() {
        const reveals = document.querySelectorAll('.reveal');

        const revealOptions = {
            threshold: 0.3,
            rootMargin: "0px 0px -50px 0px"
        };
```

```
    const revealOnScroll = new IntersectionObserver(function(entries, observer) {
        entries.forEach(entry => {
            if (!entry.isIntersecting) return;
            entry.target.classList.add('active');
            // Optional: Stop observing once revealed
            // observer.unobserve(entry.target);
        });
    }, revealOptions);

    reveals.forEach(reveal => {
        revealOnScroll.observe(reveal);
    });

    // Trigger first element immediately
    setTimeout(() => {
        reveals[0].classList.add('active');
    }, 100);
}


/* --- 2026 Modern Interactive Canvas Engine (Stars + Fireworks) --- */
const canvas = document.getElementById('ambient-canvas');
const ctx = canvas.getContext('2d');

let width, height;
let particles = [];
let fireworks =[];
let mouse = { x: 0, y: 0 };

function resize() {
```

```
    width = canvas.width = window.innerWidth;
    height = canvas.height = window.innerHeight;
}
window.addEventListener('resize', resize);
resize();

window.addEventListener('mousemove', (e) => {
    mouse.x = e.clientX;
    mouse.y = e.clientY;
});

// Ambient Star Particles
class Star {
    constructor() {
        this.x = Math.random() * width;
        this.y = Math.random() * height;
        this.size = Math.random() * 1.5;
        this.baseAlpha = Math.random() * 0.5 + 0.1;
        this.speedY = Math.random() * -0.5 - 0.1;
        this.speedX = (Math.random() - 0.5) * 0.3;
    }
    update() {
        this.y += this.speedY;
        this.x += this.speedX;

        // Parallax mouse effect
        let dx = mouse.x - this.x;
        let dy = mouse.y - this.y;
        let distance = Math.sqrt(dx * dx + dy * dy);
        if (distance < 100) {
            this.x -= dx * 0.01;
```

```
            this.y -= dy * 0.01;
        }

        if (this.y < 0) {
            this.y = height;
            this.x = Math.random() * width;
        }
    }
    draw() {
        ctx.fillStyle = `rgba(255, 255, 255, ${this.baseAlpha})`;
        ctx.beginPath();
        ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);
        ctx.fill();
    }
}

for (let i = 0; i < 150; i++) {
    particles.push(new Star());
}

// Firework Particle
class FireworkParticle {
    constructor(x, y, color) {
        this.x = x;
        this.y = y;
        this.color = color;
        const angle = Math.random() * Math.PI * 2;
        const speed = Math.random() * 6 + 2;
        this.vx = Math.cos(angle) * speed;
        this.vy = Math.sin(angle) * speed;
        this.life = 1.0;
```

```
        this.decay = Math.random() * 0.015 + 0.015;
        this.size = Math.random() * 3 + 1;
    }
    update() {
        this.x += this.vx;
        this.y += this.vy;
        this.vy += 0.05; // gravity
        this.life -= this.decay;
        this.size *= 0.96;
    }
    draw() {
        ctx.globalAlpha = this.life;
        ctx.fillStyle = this.color;
        ctx.beginPath();
        ctx.arc(this.x, this.y, this.size, 0, Math.PI * 2);
        ctx.fill();
        ctx.globalAlpha = 1;
    }
}

function createFirework(x, y) {
    const colors =['#8a2be2', '#ff2a85', '#00f3ff', '#ffffff'];
    const color = colors[Math.floor(Math.random() * colors.length)];
    for (let i = 0; i < 60; i++) {
        fireworks.push(new FireworkParticle(x, y, color));
    }
}

// Animation Loop
function animate() {
    // Slight trail effect
```

```
    ctx.fillStyle = 'rgba(5, 5, 7, 0.2)';
    ctx.fillRect(0, 0, width, height);

    // Update & Draw Stars
    particles.forEach(p => {
        p.update();
        p.draw();
    });

    // Update & Draw Fireworks
    for (let i = fireworks.length - 1; i >= 0; i--) {
        let f = fireworks[i];
        f.update();
        f.draw();
        if (f.life <= 0) {
            fireworks.splice(i, 1);
        }
    }

    requestAnimationFrame(animate);
}
animate();

/* --- "Make a Wish" Interaction --- */
const wishBtn = document.getElementById('wish-btn');
wishBtn.addEventListener('click', (e) => {
    const rect = wishBtn.getBoundingClientRect();
    const btnX = rect.left + rect.width / 2;
    const btnY = rect.top + rect.height / 2;

    // Central explosion
```

```
        createFirework(btnX, btnY);

        // Random screen explosions
        setTimeout(() => createFirework(width * 0.2, height * 0.3), 300);
        setTimeout(() => createFirework(width * 0.8, height * 0.4), 600);
        setTimeout(() => createFirework(width * 0.5, height * 0.2), 900);
        setTimeout(() => createFirework(width * 0.3, height * 0.6), 1200);
        setTimeout(() => createFirework(width * 0.7, height * 0.7), 1500);

        wishBtn.innerText = "Wish Granted 💖 ";
        wishBtn.style.pointerEvents = "none";
        wishBtn.style.borderColor = "#ff2a85";
        wishBtn.style.color = "#ff2a85";
      });
    </script>
</body>
</html>
```